

به نام خدا

۸۱۰۱۰۴۳۸۱

پایانترم برنامه سازی پیشرفته

متین حاجی دولو

۱.

مشکلات مدیریت حافظه:

متغیرهای گلوبال از ابتدای شروع برنامه تا پایان در حافظه باقی می‌مونند در حالی که متغیرهای لوکال تنها در بلوک مربوط به خودشان اجرا و پاک می‌شوند. این مورد باعث مصرف بی‌مورد دائمی حافظه می‌شود و در پروژه‌های بزرگتر با ساختارهای داده بزرگ مشکل می‌سازد.

نقض کپسوله سازی و سختی دیباگ: اگر خطایی به خاطر تغییر مقدار متغیر گلوبال که از همه جای برنامه قابل تغییر هست به وجود بیاید، پیدا کردن اون قطعه کد زمانبر هست و اصل کپسوله سازی نقض میشه و بسیار سخت میشود دیباگ کرد.

تداخل اسم‌ها: اگر از چند نیم اسپیس یا فضای نام استفاده کنیم متغیر در مقیاس گلوبال باعث تداخل اسم‌ها در برنامه و در نهایت باگ و خطاهای مختلف کامپایل می‌شود.

```
long long alternate_digit_sum(long long val, bool
is_addition = true) {
    if (val == 0) {
        return 0;
    }

    int current_digit = val % 10;

    if (is_addition) {
        return current_digit +
alternate_digit_sum(val / 10, false);
    }
    else {
        return -current_digit +
alternate_digit_sum(val / 10, true);
    }
}
```

۳.

۱. در بلوک کد ۱۹ تا ۲۱ کلاس فرزند "student" می خواهد از کلاس پدر "person" نمونه سازی شود . کلاس پدر خودش سازنده دارد پس قبل اجرای بدنه سازنده student کامپایلر سعی میکند سازنده پیش فرض پدر را صدا بزند که وجود ندارد پون وقتی کلاس "person" سازنده پارامتر دار دارد دیگر کامپایلر برایش سازنده پیش فرض نمی سازد .

نحوه رفع:

برای پاس دادن مقدار به سازنده گلاس پدر مقدار دهی اولیه را به صورت زیر انجام می دهیم:

```
Student(string _name, string _nin) :
```

```
Person(_name, _nin) {}
```

۲.

چند جا دسترسی غیرمجاز به متغیرهای "private" داریم . در خط های ۲۰ و ۲۱ و ۳۴ به خصوص . متغیرهای name , nin در کلاس "person" پرایوتند و کلاس فرزند "student" نباید دسترسی بگیرد.

رفع :

اگر بخواهیم دسترسی ایجاد شود در خط ۶ میتوانیم قبل از تعریف متغیرها بنویسیم:

Protected:

3.

تابع "print" در خط ۳۳ به صورت const تعریف شده که وضعیت شی تغییر نکند اما در خط ۳۵ ام تابع total را صدا میزند که در خط ۲۷ به صورت غیر "const" تعریف شده پس کامپایلر فرض میکند ممکن است شی را تغییر دهد و اجازه نمی دهد که یک تابع "const" یک تابع غیر "const" را فراخوانی کند.

رفع:

تابع

Total هم به صورت "کانست" بنویسیم:

```
double total() const {}
```

۴.

اسم وکتوری که در خط ۳۸ تعریف شده grades

هست امدار خط ۲۴ نوشته شده:

```
grade.push_back(g);
```

که کامپایلر آن را نمی شناسد.

نحوه رفع:

```
grades.push_back(g);
```

۵. در خط ۱۳ به جای == از = استفاده شده که خروجی را اشتباه میکند چون خروجی تابع bool است

و با این کد مقدار p.name درون name ریخته میشود که یک رشته است.

نحوه رفع:

```
return name == p.name;
```

۶. در خط ۱۲ ورودی تابع به صورت رفرنس Person& تعریف شده کامپایلر اما با توجه به خط ۴۴ که

کلمه new پوینتر *Student را برمیگرداند اجازه نمی دهد که پوینتر را به جای رفرنس پاس دهیم

نحوه رفع:

کلمه new حذف شود و در ادامه ورودی خط ۱۲ هم const شود.

```
int main() {  
    prepare();  
    bool is_fatal = false;  
  
    try {  
        while (true) {  
            try {  
                process();  
                break;  
            }  
            catch (const ErrorEx&) {  
            }  
        }  
    }  
    catch (const FatalEx&) {  
        is_fatal = true;  
    }  
  
    clean_up();  
  
    if (is_fatal) {  
        return -1;  
    }  
}
```

```
}  
  
return 0;  
}
```

۵.

گزینه "ث"

خطای زمان اجرا: دسترسی غیرمجاز به حافظه

در خط مربوط به :

$y = x$;

کانستراکتور یا سازنده کپی فراخوانی شده که در بدنه آن برخلاف سازنده پیش فرض هیچ حافظه ای

برای

Data

در شی جدید آن تخصیص داده نشده است که باید با دستور

New

نوشته میشود. در نتیجه حلقه

For

تلاش میکند که مقادیر را در جای نامشخصی کپی کند که منجر به

Segmentation Fault

میشود.

۶.

(الف)

در کلاس Reservation اشاره گر *Customer تعریف شده در حالی که در ادامه کلاس آن تعریف می شود و کامپایلر هنوز نمیداند که چیست پس خطای کامپایل خواهد داد.

نحوه رفع:

میتوانیم قبل از کلاس Reservation کلاس Customer را دیکلر و تعریف کنیم و بعد در ادامه همانجا کد باقی بماند:

```
class Customer;  
class Reservation {...};
```

(ب)

برای اینکه اطلاعات مشتریان در کل برنامه هماهنگ باشد بهتر است به جای pass by value از pass by reference استفاده شود تا اگر اطلاعات یک مشتری در جایی تغییر کرد این تغییر در لیست system تغییر کند و به یک آدرس واحد در heap اشاره کند. اگر از وکتور استفاده میکردیم تغییر سایز وکتور سرعت کمتری برای کپی و انتقال نسبت به اشاره گر به ما میداد و موضوع اشاره شده در ابتدا هم برای وکتور مشکل ساز میشد.

(ج)

نقطه ضعف از وجه درستی کارکرد: هیچ کلاس مرکزی کنترل مستقیم برا پروازها ندارد و اگر مثلاً بخواهیم یک پرواز را حذف کنیم، پیدا کردن و آزادسازی آن پرواز از حافظه سخت است. تازه با حذف آن پرواز در کلاس Reservation اشاره گر Flight معلق میماند و دسترسی به آن احتمالاً مشکل ساز میشود.

نقطه ضعف از وجه سرعت اجرا: برای دسترسی به پروازها باید تمام مشتری ها پیمایش شود بعد تمام رزروها خوانده شود تا لیست پروازها استخراج شود که راه مستقیمی برای دسترسی به پروازها

نیست که این پیچیدگی باعث کاهش سرعت در زمان طولانی میشود در صورتی که با یک لیست مرکزی در system این دسترسی را در زمانب بسیار کوتاه ممکن میکند.

(د)

ابتدا در کلاس system یک حلقه for روی customers میزنیم و داخل آن یک حلقه for دیگر روی وکتور reservation آن مشتری بزنیم و بررسی کنیم آیا *flight آن پرواز با پرواز ما برابر است یا نه که اگر بود مشتری به لیست نتایج اضافه شود.

پیشنهاد: میتوانیم یک وکتور از مشتریان یا رزروها به کلاس Flight اضافه کنیم تا مسافران پرواز شناسایی شوند:

```
class Flight {  
    vector<Reservation*> reservations;  
};
```

تبعات منفی: نیاز است لیست داخل Customer و Flight همزمان اپدیت شوند که رابطه ای دوطرفه است و ممکن است عدم هماهنگی برای اپدیت مشکل ساز شود.


```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <algorithm>

using namespace std;

const int MIN_TIME_VALUE = 0;
const int MAX_HOUR_VALUE = 23;
const int MAX_MINUTE_VALUE = 59;
const int MIN_CLASS_DURATION = 30;
const int MINUTES_IN_HOUR = 60;
const int ERROR_EXIT_CODE = 1;

enum DayOfWeek {
    SAT, SUN, MON, TUE, WED, THU, FRI
};

class Time {
    int total_minutes;

public:
    Time(int hour, int minute) {
        if (hour < MIN_TIME_VALUE || hour > MAX_HOUR_VALUE ||
            minute < MIN_TIME_VALUE || minute > MAX_MINUTE_VALUE) {
```

```

        exit(ERROR_EXIT_CODE);
    }

    total_minutes = (hour * MINUTES_IN_HOUR) + minute;
}

int calculate_difference(const Time& other_time) const {
    return total_minutes - other_time.total_minutes;
}

bool is_less_or_equal(const Time& other_time) const {
    return total_minutes <= other_time.total_minutes;
}

bool is_strictly_less(const Time& other_time) const {
    return total_minutes < other_time.total_minutes;
}
};

class TimeSlot {
    int week_day;
    Time start_time;
    Time end_time;

public:
    TimeSlot(int day, int start_h, int start_m, int end_h, int
end_m)

```

```

        : week_day(day), start_time(start_h, start_m),
end_time(end_h, end_m) {

        bool is_invalid_duration =
end_time.calculate_difference(start_time) <
MIN_CLASS_DURATION;

        if (day == FRI ||
!start_time.is_strictly_less(end_time) || is_invalid_duration)
{

                exit(ERROR_EXIT_CODE);

        }

}

bool check_overlap(const TimeSlot& other_slot) const {

        if (week_day != other_slot.week_day) {

                return false;

        }

        return
!(end_time.is_less_or_equal(other_slot.start_time) ||
other_slot.end_time.is_less_or_equal(start_time));

}

};

class WeeklySchedule {

        vector<TimeSlot> scheduled_slots;

        void validate_no_internal_conflict(const TimeSlot&
new_slot) const {

                if (has_conflict_with_slot(new_slot)) {

                        exit(ERROR_EXIT_CODE);

```

```

    }

}

public:

    void add_time_slot(int day, int start_h, int start_m, int
end_h, int end_m) {

        TimeSlot new_slot(day, start_h, start_m, end_h,
end_m);

        validate_no_internal_conflict(new_slot);

        scheduled_slots.push_back(new_slot);

    }

    bool has_conflict_with_slot(const TimeSlot& external_slot)
const {

        auto overlap_predicate = [&](const TimeSlot& slot) {

            return slot.check_overlap(external_slot);

        };

        return std::any_of(scheduled_slots.begin(),
scheduled_slots.end(), overlap_predicate);

    }

    bool conflicts(const WeeklySchedule& other_schedule) const
{

        auto conflict_predicate = [&](const TimeSlot& slot) {

            return

other_schedule.has_conflict_with_slot(slot);

        };

        return std::any_of(scheduled_slots.begin(),
scheduled_slots.end(), conflict_predicate);

```

```
    }  
};
```